

Object Oriented Concepts using C++

Course Topics

- Introduction to Object-Oriented Programming
 - Introduction to C++
 - Namespaces
 - Classes and Objects
 - Operations
 - Exception Handling
 - Memory Management
 - Advanced Features of C++
 - Inheritance
 - Virtual Functions
-

Day 1 Agenda

- POP vs OOP
 - Limitations of C
 - History of C++
 - Object-Oriented Programming
 - Data Types
 - Structures in C and C++
-

Limitations of C

#	Limitation	Details
1	No Data Security	C has no access control mechanism — any function can access and modify any data, making it unsafe for large applications.
2	Hard to Manage Complex Projects	As project size grows, C code becomes difficult to organize, maintain, and debug due to the lack of modularity.
3	Limited Function Reusability	C does not support classes or inheritance, so code reuse is achieved only through functions — which is limited compared to OOP.

History of C++

Year	Event
—	Simula — World's first OOP language (introduced classes and objects)
—	C — Fastest procedural language, but lacked OOP features

Year	Event
1979	Bjarne Stroustrup began work on " <i>C with Classes</i> " — combining C's speed with OOP concepts
1983	Renamed to C++ (C Plus Plus) — the ++ symbolizes an increment/improvement over C

💡 **Bjarne Stroustrup** (Bell Labs) is the inventor of C++.

Object-Oriented Programming (OOP)

A language is called **Object-Oriented** if it follows the core OOP pillars:
Abstraction, Encapsulation, Modularity, Hierarchy, and Polymorphism.

Major Pillars (Must Have)

Pillar	Description
Abstraction	Hiding internal implementation details and showing only the necessary features to the user.
Encapsulation	Binding data and the functions that operate on it into a single unit (class), and restricting direct access.
Modularity	Breaking a program into smaller, independent, manageable modules/classes.
Hierarchy	Organizing classes in a parent-child relationship — includes Inheritance and Association .

Minor Pillars (Good to Have)

Pillar	Description
Polymorphism	The ability of a function or object to behave differently in different contexts (compile-time & run-time).
Concurrency	Ability to handle multiple tasks/threads simultaneously.
Persistence	Ability of an object to retain its state beyond the program's lifetime (e.g., saved to a file or database).

Data Types

A **data type** defines three things about a variable:

Property	Meaning
Memory	How much memory is required to store the data
Nature	What kind of data can be stored (integer, decimal, character, etc.)
Operations	What operations can be performed on that data

1. Fundamental / Built-in Types

These are provided directly by the language.

Type	Size (typical)	Nature
<code>int</code>	4 bytes	Whole numbers
<code>float</code>	4 bytes	Decimal numbers (single precision)
<code>double</code>	8 bytes	Decimal numbers (double precision)
<code>long</code>	4/8 bytes	Large whole numbers
<code>char</code>	1 byte	Single character
<code>bool</code>	1 byte	<code>true</code> or <code>false</code>
<code>wchar_t</code>	2/4 bytes	Wide character (Unicode / multilingual characters)

💡 `bool` and `wchar_t` are additions in C++ — not present in standard C.

2. Derived / User-Defined Types

These are built using fundamental types.

Type	Description
<code>array</code>	Collection of elements of the same type
<code>string</code>	Sequence of characters
<code>struct</code>	Groups variables of different types under one name
<code>union</code>	Similar to struct, but all members share the same memory location
<code>pointer</code>	Stores the memory address of another variable
<code>function</code>	A named block of code that performs a specific task

`bool` — Boolean Type

- Stores only two values: `true` (1) or `false` (0)
- Size: **1 byte**
- Used in conditions, loops, and flags

```
bool isPassed = true;
bool isFailed = false;
cout << isPassed; // Output: 1
```

`wchar_t` — Wide Character Type

- Stands for **Wide Character Type**
- Used to store **Unicode / multilingual characters** that cannot fit in a single `char` (1 byte)
- Size: **2 bytes** (Windows) or **4 bytes** (Linux/Unix)
- Use `L` prefix for wide character literals

```
wchar_t ch = L'A';           // Wide character
wchar_t name[] = L"Hello";   // Wide string
wcout << ch;                 // Use wcout (not cout) for wide characters
```

💡 Use `wcout` instead of `cout` when printing `wchar_t` values.
